

# INTEGRATIVE 2025

## Semester B

### Project Requirements Document - SoldiCare

10143

**Department:** Software Engineer

**Lecturers:** Deganit Armon, Tom Cohen

**Students:**

- Itay Chabra  
Role: Team, Team Lead, DBA
- Idan Noyshul  
Role: Team, Devops , UI/UX , System Architecture
- Sapir Gilany  
Role: Team, Scrum Master, Tech Writer , UI/UX , System Architecture
- Neta Ben Mordechai  
Role: Team, Product Owner
- Roi Dor  
Role: Team, QA Engineer

## Table of Contents

|     |                                             |    |
|-----|---------------------------------------------|----|
| 1.  | Introduction.....                           | 3  |
| 1.1 | Purpose of System.....                      | 3  |
| 1.2 | Scope of System.....                        | 4  |
| 2.  | Actors and goals.....                       | 5  |
| 2.1 | Soldiers (Primary Actor).....               | 5  |
| 2.2 | Military Medics (Primary Actor) .....       | 5  |
| 2.3 | Commanders (Primary Actor) .....            | 5  |
| 2.4 | Medical Command Center (Support Actor)..... | 6  |
| 3.  | Functional Requirements .....               | 6  |
| 3.1 | Use Case Diagram.....                       | 6  |
| 3.2 | Use Case Details .....                      | 6  |
| 4.  | Non-Functional Requirements .....           | 9  |
| 5.  | Appendix - Technologies .....               | 10 |
| 5.1 | Server Side .....                           | 10 |
| 5.2 | Frontend Web.....                           | 10 |
| 5.3 | Image Storage .....                         | 10 |
| 5.4 | Sensor Simulation .....                     | 11 |
| 5.5 | Android Application .....                   | 11 |
| 6.  | Appendix – Project Summary .....            | 13 |
| 6.1 | Kanban Board Samples .....                  | 13 |
| 6.2 | Project Retrospective .....                 | 22 |
| 7.  | Appendix – External Links .....             | 25 |
| 7.1 | Figma .....                                 | 25 |
| 7.2 | System Video .....                          | 25 |
| 7.3 | Bitbucket .....                             | 25 |

## 1. Introduction

In modern combat environments, soldiers face immense physical and psychological challenges that can impact their health and operational effectiveness. Current methods for assessing soldiers' medical status in the field rely heavily on verbal communication and subjective observation, which may lead to delays in identifying critical health conditions.

SoldiCare is a smart real-time monitoring system designed to enhance medical decision-making for soldiers during combat by leveraging wearable sensors and cutting-edge Ambient Invisible Intelligence technology.

The concept of Ambient Invisible Intelligence, first defined by Gartner in late 2024, refers to the seamless integration of intelligent sensors that unobtrusively collect and analyze real-time data. In the context of SoldiCare, this technology ensures continuous health monitoring without interfering with soldiers' operational performance. By utilizing non-intrusive sensors, crucial biometric data such as blood pressure, heart rate, and oxygen levels can be gathered effortlessly, providing essential insights into soldiers' well-being in real time.

The development of SoldiCare aims to provide military units with access to accurate and timely medical data, supporting the early detection of health issues and enabling faster response times in combat scenarios. By integrating real-time health monitoring into operational decision-making, the system has the potential to enhance medical readiness and improve soldiers' well-being in the field.

### 1.1 Purpose of System

SoldiCare is designed to provide real-time health monitoring for soldiers in combat environments, enabling early detection of medical issues and data-driven decision-making. The system addresses the critical need for accurate, immediate medical insights that support operational effectiveness and soldier safety.

Beyond individual monitoring, SoldiCare also aims to provide a broader situational overview of the health status across entire military units. By analyzing aggregated health data, commanders can assess the combat readiness of their forces and determine whether adjustments are needed for mission planning. Instead of relying on "whoever shouts the loudest on the radio," this system will assist in prioritizing medical resource allocation, ensuring that aid is dispatched efficiently to the areas where it is needed most.

The primary goal of SoldiCare is to enhance battlefield medical readiness, improve resource allocation for emergency response, and support proactive intervention to prevent medical crises. By delivering objective, real-time health data, the system empowers decision-makers to prioritize treatment efficiently and respond effectively to evolving medical situations.

## **1.2 Scope of System**

SoldiCare operates as a medical monitoring system tailored for battlefield environments, providing continuous assessment of soldiers' health status. The system's scope includes:

### *1.2.1 Core Functionality*

- Collection of biometric data – Monitoring heart rate, blood pressure, oxygen saturation, stress indicators, and additional physiological metrics.
- Real-time data transmission – Displaying collected health data and unit readiness on the system's statistics dashboard, providing immediate visibility for medical units and command centers.
- Automated alerts for abnormal health patterns – Detecting irregularities and triggering real-time warnings, supporting quick decision-making in emergency scenarios.
- User and admin role management – Defining access levels to ensure proper handling and monitoring of medical data across different system users.

### *1.2.2 Security and Data Protection*

- No Guaranteed Encryption for Transmitted Data – The system does not provide full encryption for transmitted biometric data, and there is no assurance of secure end-to-end data protection.
- Limited Access Control Mechanisms – While user roles are defined, there are no advanced authentication or authorization layers to restrict access to sensitive information.
- No Dedicated Cybersecurity Monitoring – The system does not include active security monitoring or intrusion detection to identify unauthorized access or potential threats.

### *1.2.3 Operational Constraints*

- Designed for combat conditions but relies on stable network connectivity for real-time updates.
- The system does not replace direct medical evaluations but supplements them with objective health indicators.
- The system assumes that wearable sensors maintain constant connectivity under all operational conditions. Therefore, scenarios involving connectivity failures are not considered in the system's workflows.

### *1.2.4 Limitations*

- SoldiCare does not provide medical diagnoses but rather supports decision-making with health trends.
- Environmental factors, such as extreme weather or signal disruptions, may affect data transmission.
- The scope of this project does not include predictive AI-based medical recommendations.

## **2. Actors and goals**

### **2.1 Soldiers (Primary Actor)**

Soldiers are equipped with wearable health monitoring sensors. Their goal is to track their personal health status, receive alerts for abnormal conditions, and enhance their awareness of their physical state during combat operations.

### **2.2 Military Medics (Primary Actor)**

Field medics responsible for assessing and treating injured soldiers. Their goal is to access up-to-date medical data, quickly identify emergencies, and provide effective treatment based on accurate real-time information rather than subjective reports.

### **2.3 Commanders (Primary Actor)**

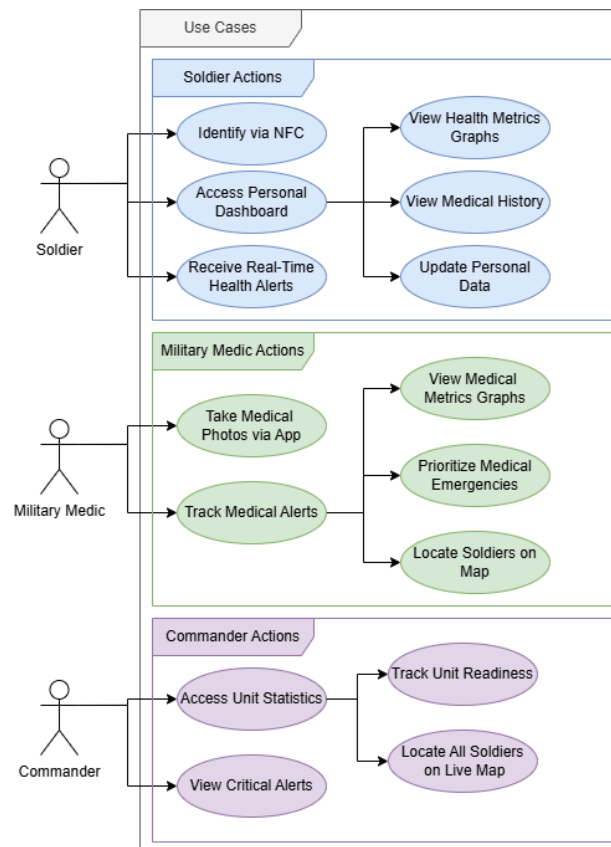
Unit commanders overseeing the combat readiness of their soldiers. Their goal is to gain a broader picture of the overall physical condition of their unit, analyze trends, and prioritize medical assistance where it is most needed. Commanders do not have access to individual medical details but can view general fitness indicators for operational planning.

## 2.4 Medical Command Center (Support Actor)

A centralized medical authority responsible for monitoring soldier health trends across multiple units. Their goal is to analyze aggregated health data, identify large-scale medical concerns, and optimize the allocation of medical resources based on battlefield conditions.

## 3. Functional Requirements

### 3.1 Use Case Diagram



### 3.2 Use Case Details

#### 3.2.1 Use Case 1

**Name:** Real-Time Health Monitoring & Alerts

**Goals:** Provide real-time tracking of soldiers' vital signs and generate alerts for medical abnormalities.

**Participating Actors:** Soldiers (Primary), Military Medics (Primary), Commanders (Primary), Medical Command Center (Support).

**Basic Workflow:**

1. Soldier's wearable sensors continuously collect and log biometric data, including heart rate, blood pressure, oxygen levels, and stress indicators.
2. Data is transmitted in real-time to the medical monitoring system.
3. The system stores all received medical metrics in the database over time and continuously processed real-time data.
4. If a critical deviation is detected (e.g., sudden drop in oxygen saturation), the system alerts the abnormality for the specific soldier.
5. Medics receive a detailed overview, including real-time graphs, previous health records, and the soldier's location for rapid intervention.

**Alternative Workflow A: False Alarm Prevention**

- A.1 At Step 4, the system detects a sudden abnormal heart rate spike but notices stabilization within seconds.
- A.2 Instead of triggering an immediate alert, the system continues monitoring to confirm the trend.
- A.3 If the soldier's health remains stable for 30 seconds, no alert is triggered, and workflow continues from Step 4.
- A.4 If further abnormalities occur, an alert is sent to the medics for investigation.

*3.2.2 Use Case 2*

**Name:** Real-Time Image Transfer

**Goals:** Enable real-time transmission of images of injured soldiers in the field by military medics.

**Participating Actors:** Military Medics (Primary), Soldiers (Primary)

**Basic Workflow:**

1. A soldier is injured in the field.
2. A medic scans the soldier's NFC tag to quickly retrieve identification details.
3. The medic captures an image of the wounded soldier, which is immediately transmitted to the system.
4. The system displays the image alongside the relevant medical alert, ensuring a rapid and informed response.

**Alternative Workflow A: Image Transmission Failure Handling**

A.1 At Step 3, multiple If the image capture fails due to network issues or device malfunction, the system automatically attempts a secondary transmission using a backup connection.

A.2 If the transmission continues to fail, the medic manually inputs key injury details into the system to ensure essential medical information is recorded.

A.3 Once communication is restored, the stored image is reprocessed and added to the soldier's medical record.

### *3.2.3 Use Case 3*

**Name:** Combat Readiness & Health Assessment

**Goals:** Provide commanders with an overview of soldiers' physical readiness for missions.

**Participating Actors:** Soldiers (Primary), Commanders (Primary), Medical Command Center (Support).

**Basic Workflow:**

1. Before an intensive mission, the commander conducts an individual and unit-wide health assessment, ensuring all soldiers are fit for deployment.
2. Soldiers report any recent injuries, fatigue, or medical concerns that might affect their combat performance.
3. The system displays each soldier's recorded medical history, allowing the commander to evaluate potential risks.
4. If necessary, the commander consults medical personnel to determine whether specific soldiers require additional rest or medical clearance.
5. Based on the assessment, final combat readiness decisions are made, ensuring optimal deployment strategy.

**Alternative Workflow A:** Extreme Physical Degradation

A.1 At Step 5, system analytics indicate a sudden decline in overall unit readiness beyond acceptable thresholds.

A.2 Commanders are alerted with predictive health deterioration metrics.

A.3 If additional rest or medical intervention is scheduled, soldier fitness levels gradually improve.

A.4 Otherwise, the unit continues monitoring trends, and workflow resumes from Step 5.



## 4. Non-Functional Requirements

A table with the following structure:

| Requirement number | Requirement Type   | Requirement Description                                                                                                                                                                                                   |
|--------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                  | U - Usability      | Visual Data Representation for Enhanced Usability – The system must display graphs and medical indicators alongside alerts to provide a clear and intuitive visualization of health trends, aiding quick decision-making. |
| 2                  | U - Usability      | Responsive System Design – The system must provide a fully responsive interface, ensuring seamless accessibility and proper display across various devices, including desktops, tablets.                                  |
| 3                  | R - Reliability    | Medical-Grade Alert Classification – The system must classify and prioritize alerts based on medical urgency standards.                                                                                                   |
| 4                  | P - Performance    | Concurrent Session Handling – The system must support at least 10 active sessions simultaneously without performance degradation, ensuring stable operation under typical workload conditions.                            |
| 5                  | S - Supportability | Extensibility & Scalability – The system must support future expansion, allowing seamless integration of additional components as needed.                                                                                 |
| 6                  | S - Supportability | Logging & Debugging Mechanisms – The system must provide comprehensive logging and error-tracking capabilities, enabling efficient fault analysis and operational maintenance.                                            |

## 5. Appendix - Technologies

### 5.1 Server Side

- Spring Boot: Used to build a modular and scalable RESTful API in Java.
- MongoDB: NoSQL database chosen for flexible document-based storage of complex, nested data structures (e.g., soldiers, sensors, alerts).
- YAML Configuration: Used to define a local development environment, including pulling a MongoDB Docker image for testing and runtime.
- Testing: Implemented behavior-driven scenarios written in Gherkin syntax and executed with JUnit.
- Render & CI/CD: Configured a CI/CD pipeline that deploys the backend to Render on each pull request, enabling seamless cloud deployment using Docker file.

### 5.2 Frontend Web

- React + TypeScript: Implemented a component-based single-page application with type safety.
- Material UI: Applied as the primary design and layout framework for responsive, accessible interfaces.
- React Hooks & Features: Utilized useState, useEffect, useContext, local storage, and .env files for state and configuration management.
- Custom Hooks: Created reusable logic abstractions to improve code maintainability and readability.
- Leaflet API: Integrated for real-time map visualization of soldier positions and alert zones.
- Recharts Library: Used to display dynamic sensor data in time-series graphs.
- Firebase API: Connected to retrieve metadata and URLs of images associated with soldier profiles.

### 5.3 Image Storage

- Firebase Storage - Cloud file storage and management
- Firebase BOM 33.15.0 - Bill of Materials for version management
- Google Services Plugin - Firebase integration

## 5.4 Sensor Simulation

- Python Script: Implemented a simulation tool using `requests` to send real-time health data to the backend.
- Spring Integration: The simulator interacts directly with the backend REST API, mimicking real-world sensor behavior.

## 5.5 Android Application

### 5.5.1 Programming Languages

- Java - Primary application logic and Android development
- Kotlin - Build configuration (Gradle build scripts)
- XML - UI layouts, resources, and Android manifest configuration

### 5.5.2 Camera & Media

- CameraX 1.3.0 - Modern camera API for image capture
- camera-camera2 - Camera2 API backend
- camera-lifecycle - Lifecycle-aware camera operations
- camera-view - Camera preview UI components
- ExifInterface 1.3.7 - Image metadata and orientation handling
- Bitmap & Graphics APIs - Image processing and compression
- FileProvider - Secure file sharing between app components

### 5.5.3 NFC Technology

- Android NFC API - Near Field Communication integration
- NDEF (NFC Data Exchange Format) - Standard for NFC data structure
- Foreground Dispatch System - Priority NFC event handling
- Multiple NFC Technologies Support:
  - NfcA (ISO 14443-3A)
  - NfcB (ISO 14443-3B)
  - NfcF (JIS 6319-4)
  - NfcV (ISO 15693)
  - NDEF (NFC Data Exchange Format)

#### *5.5.4 UI & User Experience*

- Material Design Components - Modern Android UI patterns.
- ConstraintLayout - Flexible and efficient layouts.
- Animation APIs - Visual feedback and transitions.
- Toast & Progress Indicators - User feedback systems.

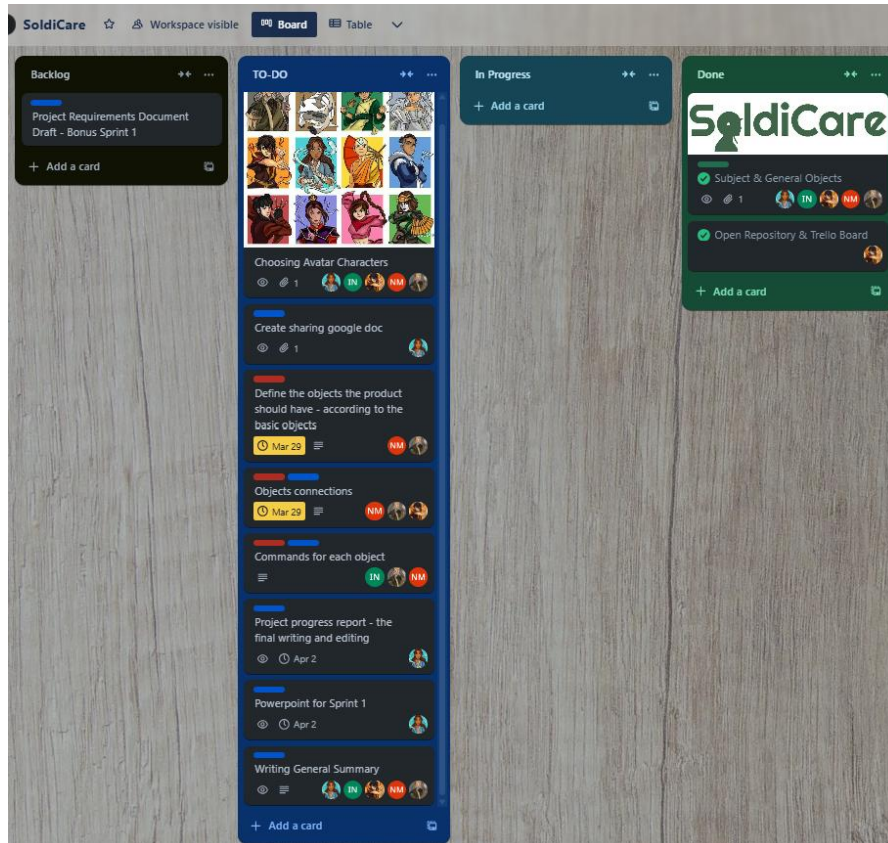
## 6. Appendix – Project Summary

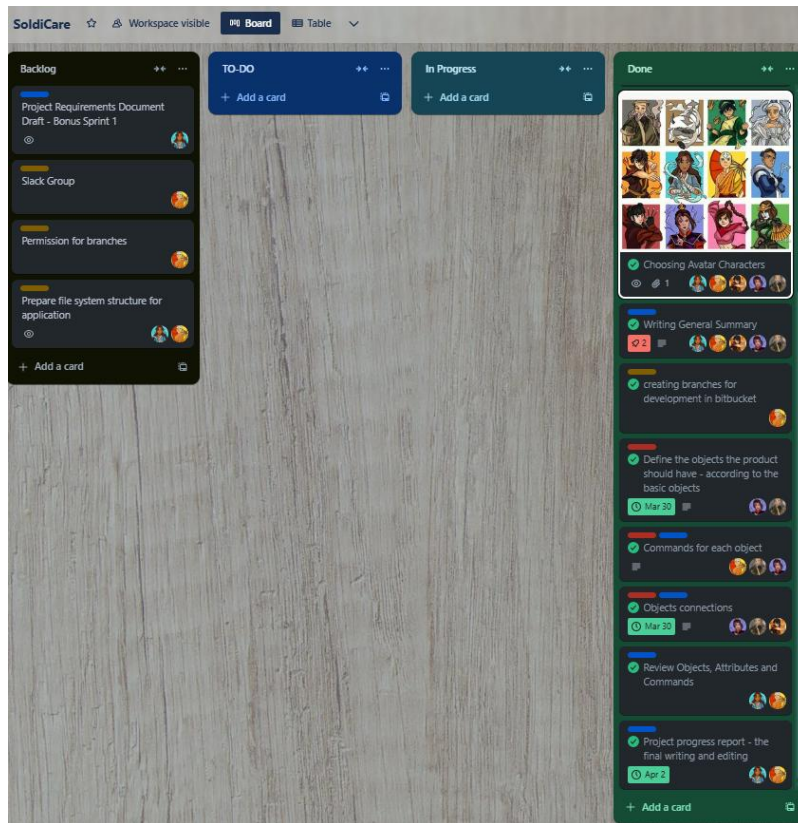
### 6.1 Kanban Board Samples

Samples collected throughout the semester are organized from the oldest to most recent, categorized by sprint.

#### 6.1.1 Sprint 1

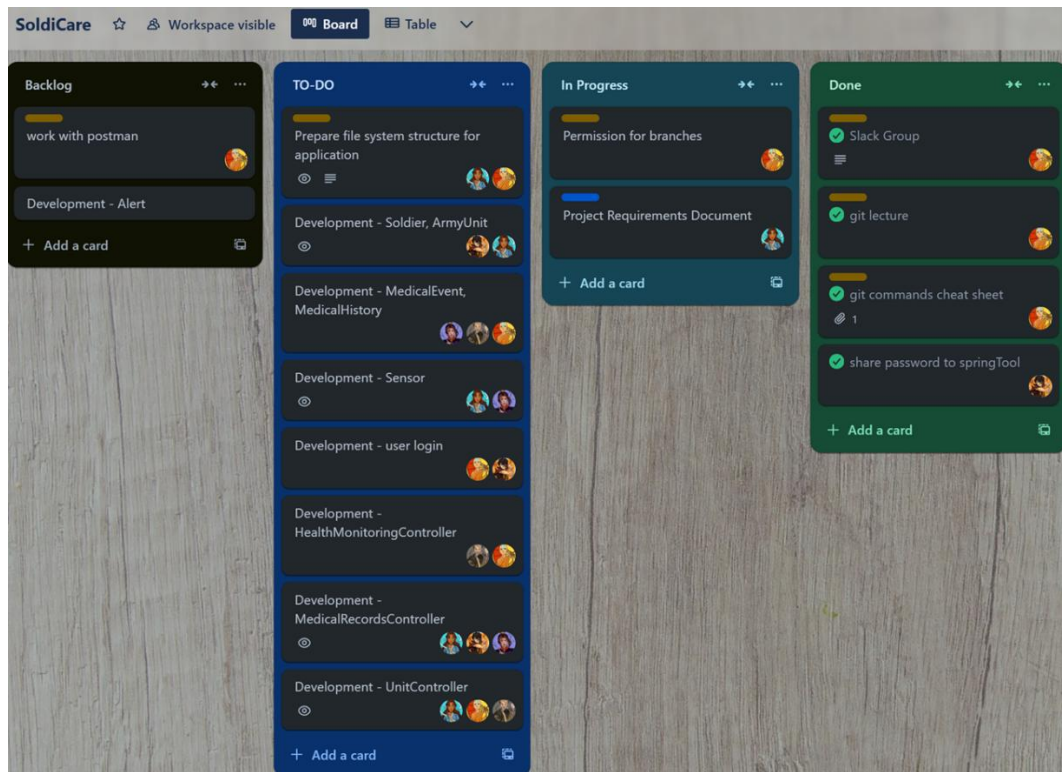
Date: 29/03/2025



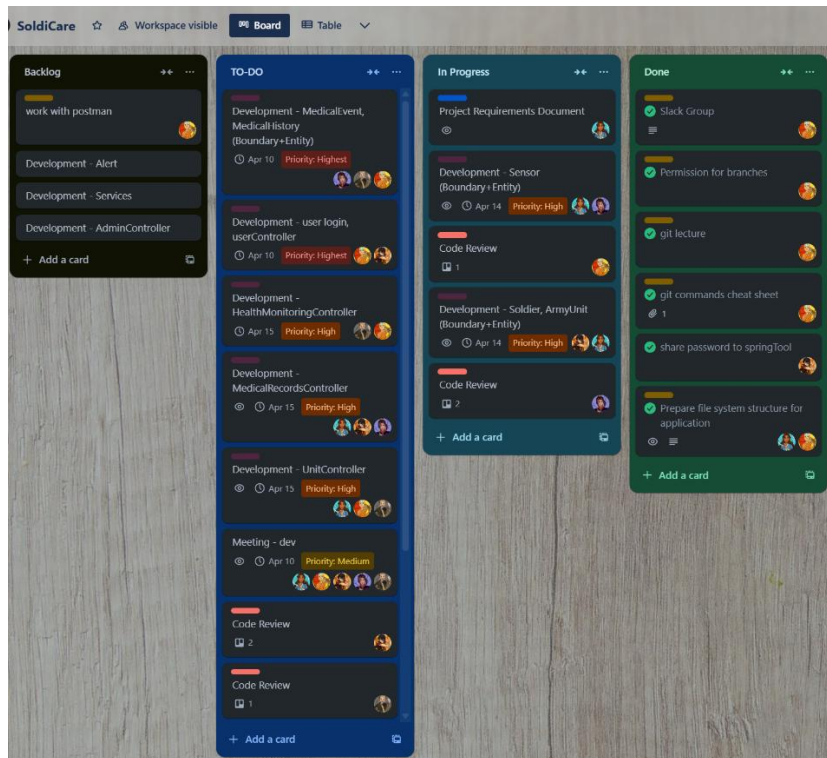


## 6.1.2 Sprint 2

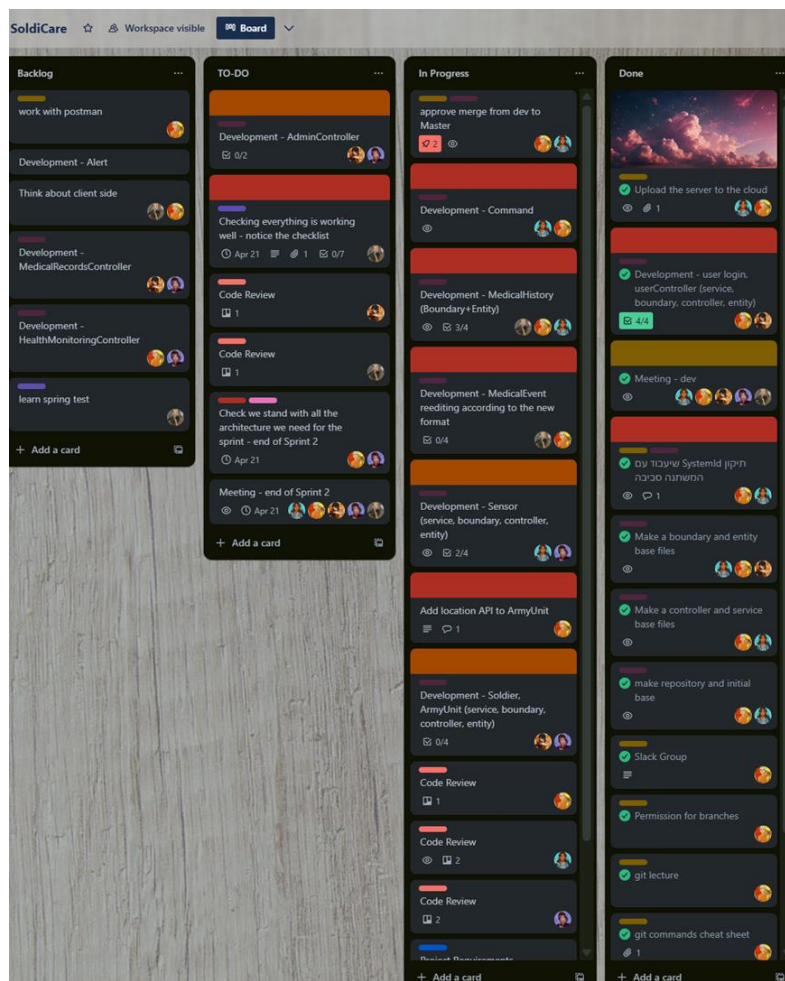
Date: 05/04/2024



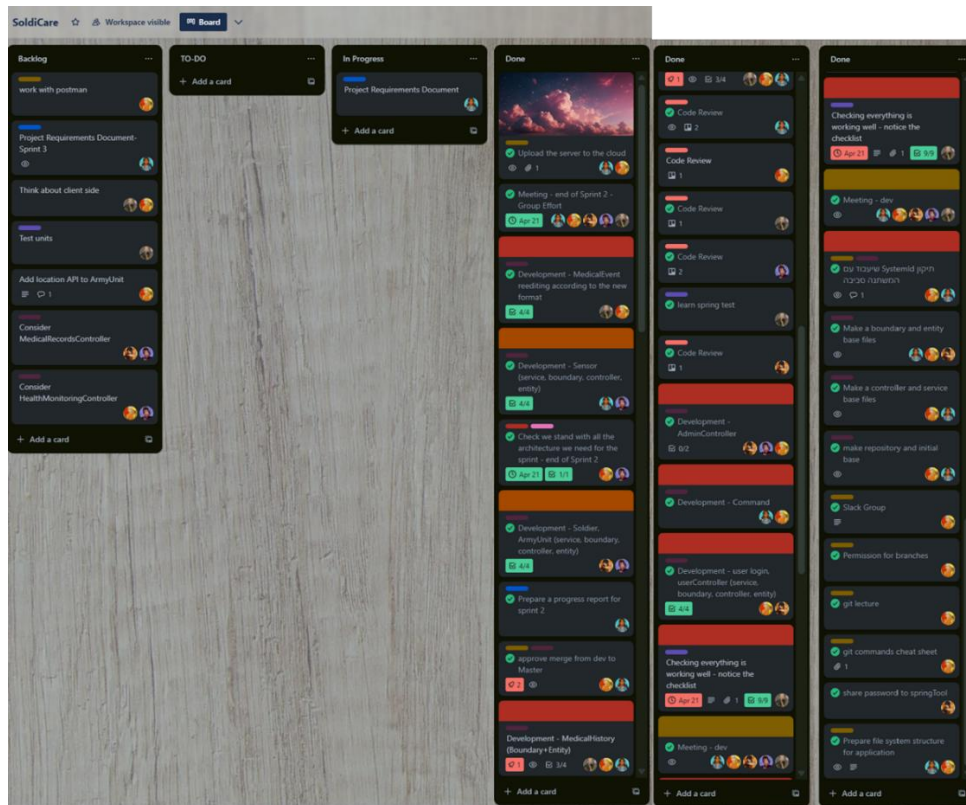




Date: 15/04/2025

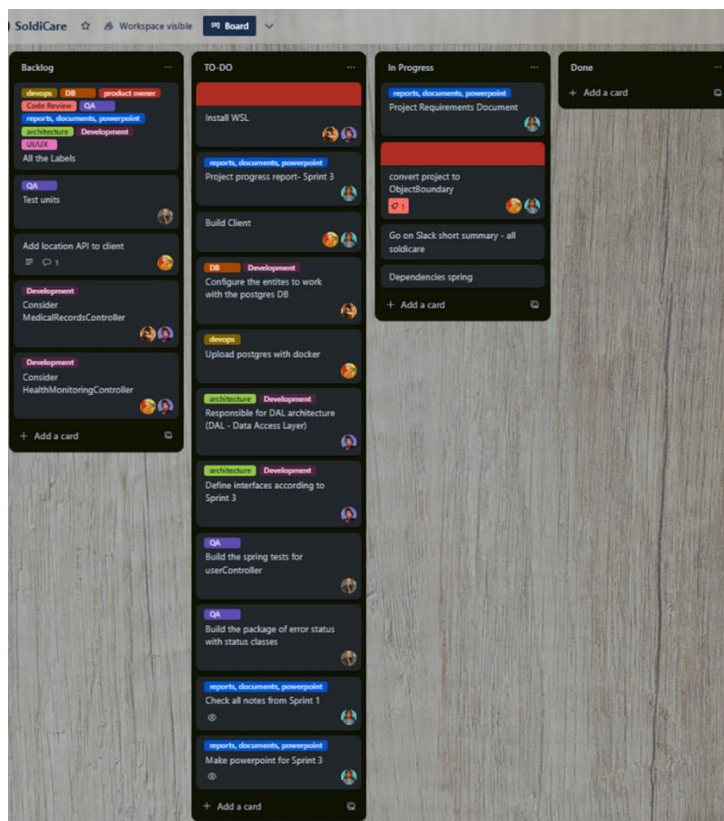


Date: 23/04/2025

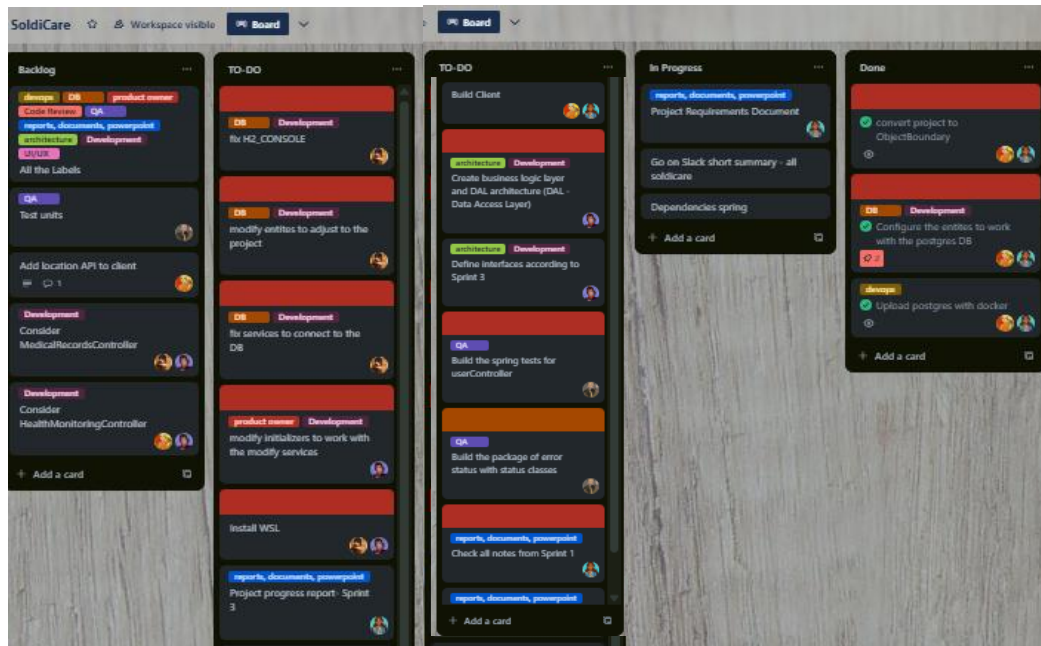


### 6.1.3 Sprint 3

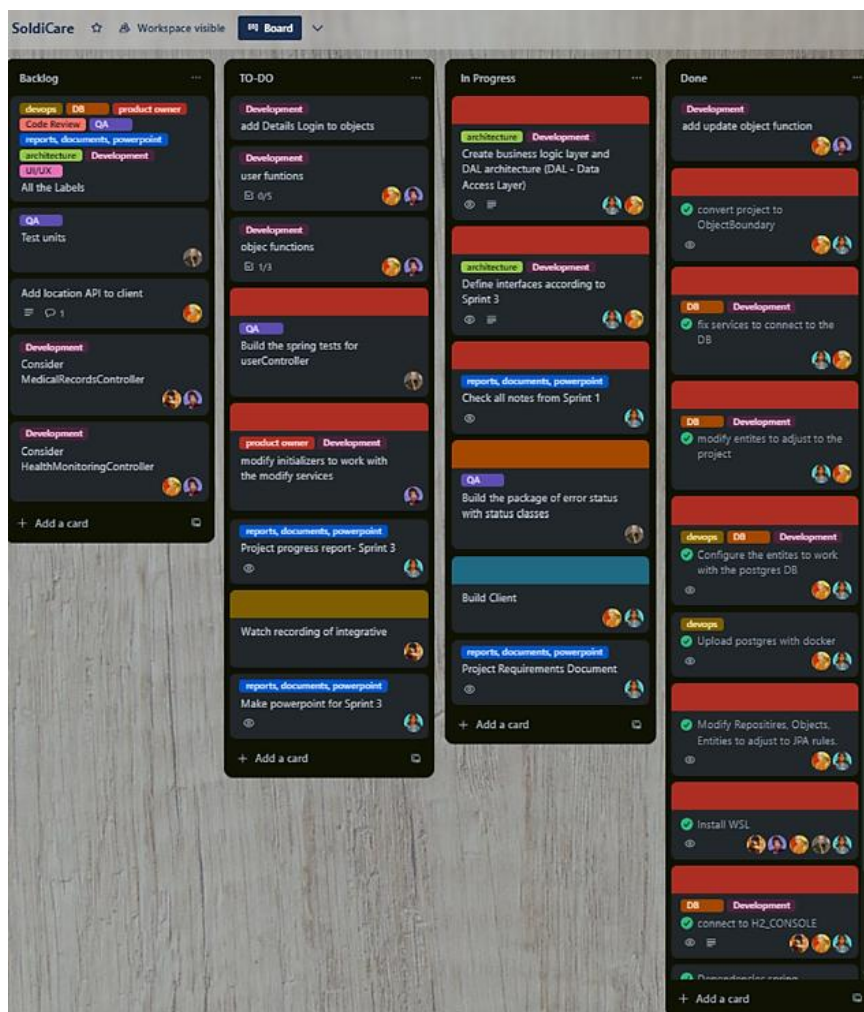
Date: 03/05/2025

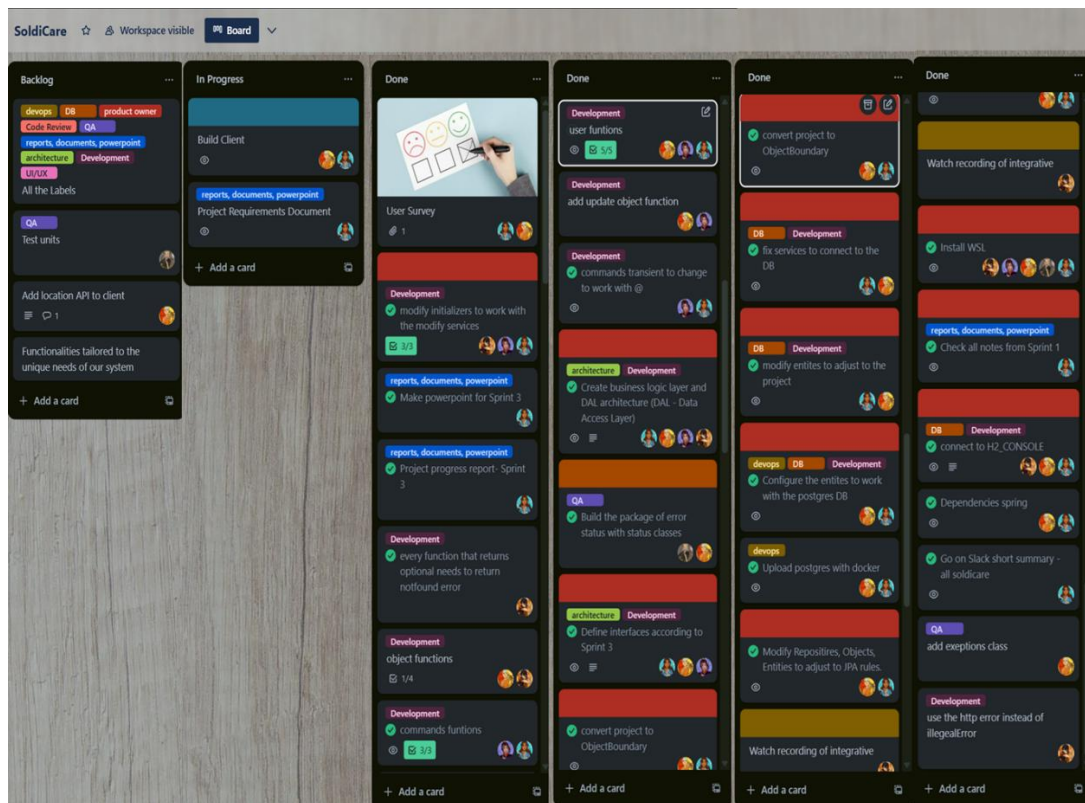






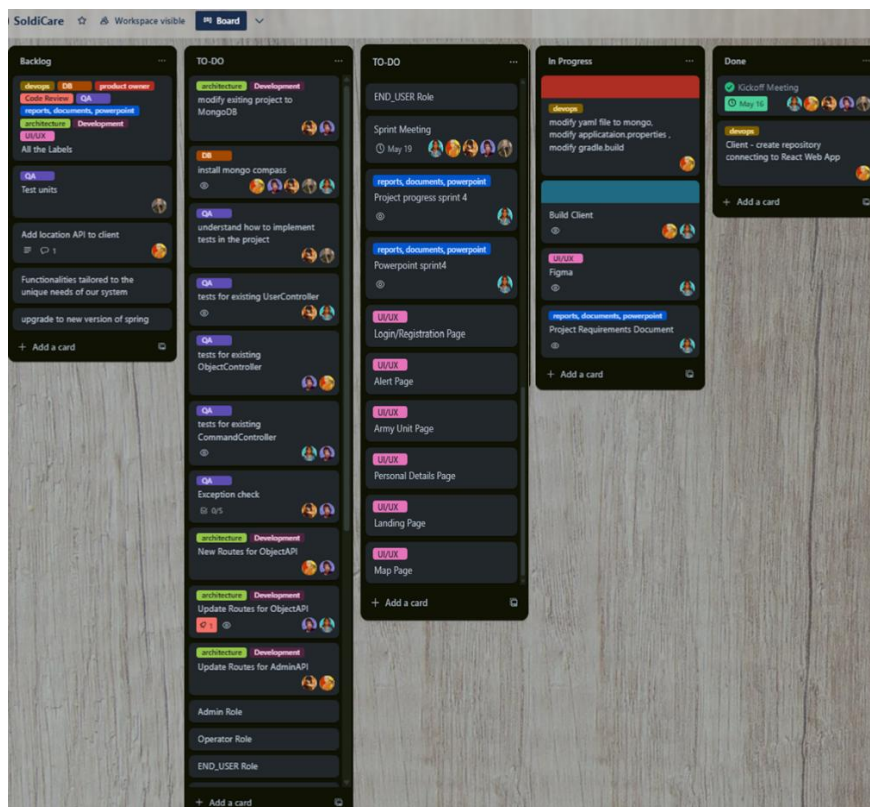
Date: 11/05/2025



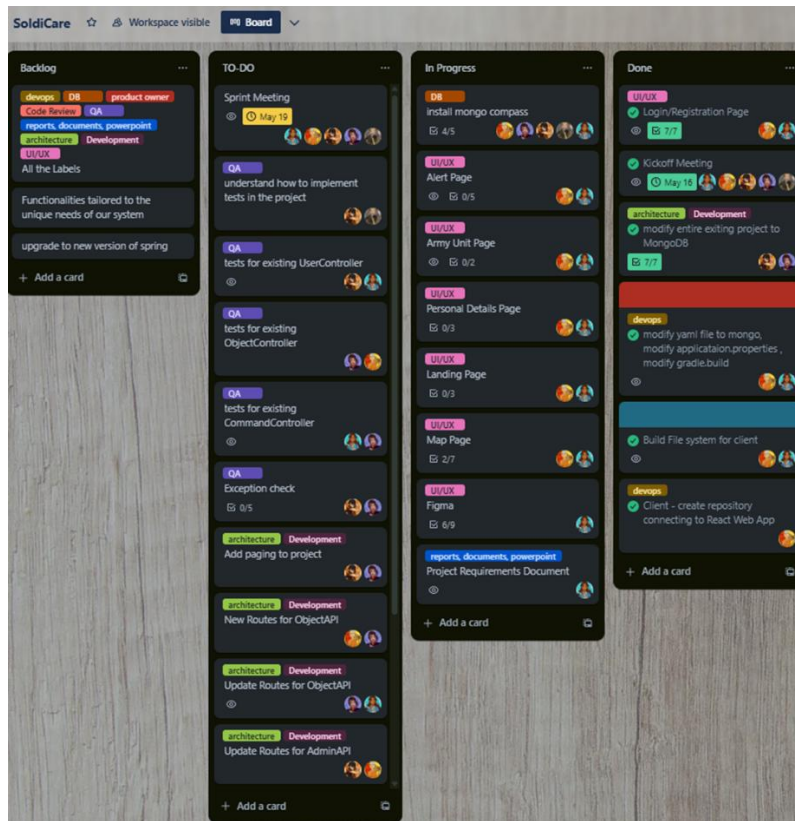


### 6.1.4 Sprint 4

Date: 16/05/2025

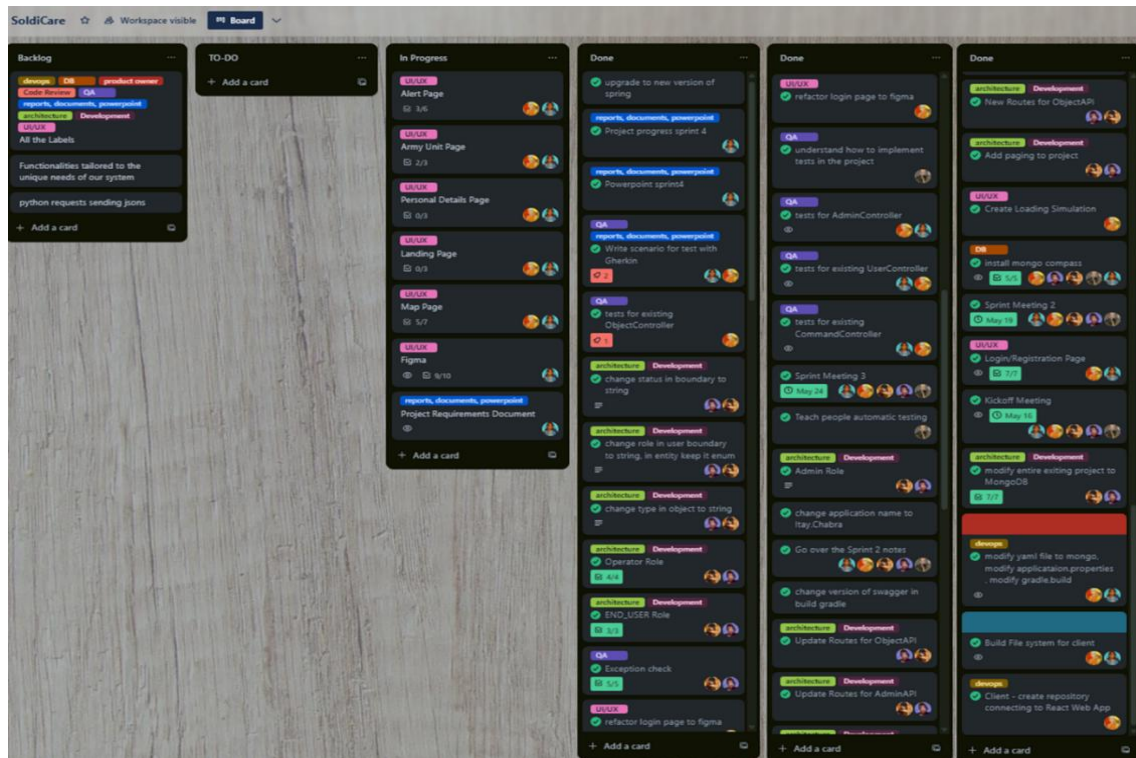






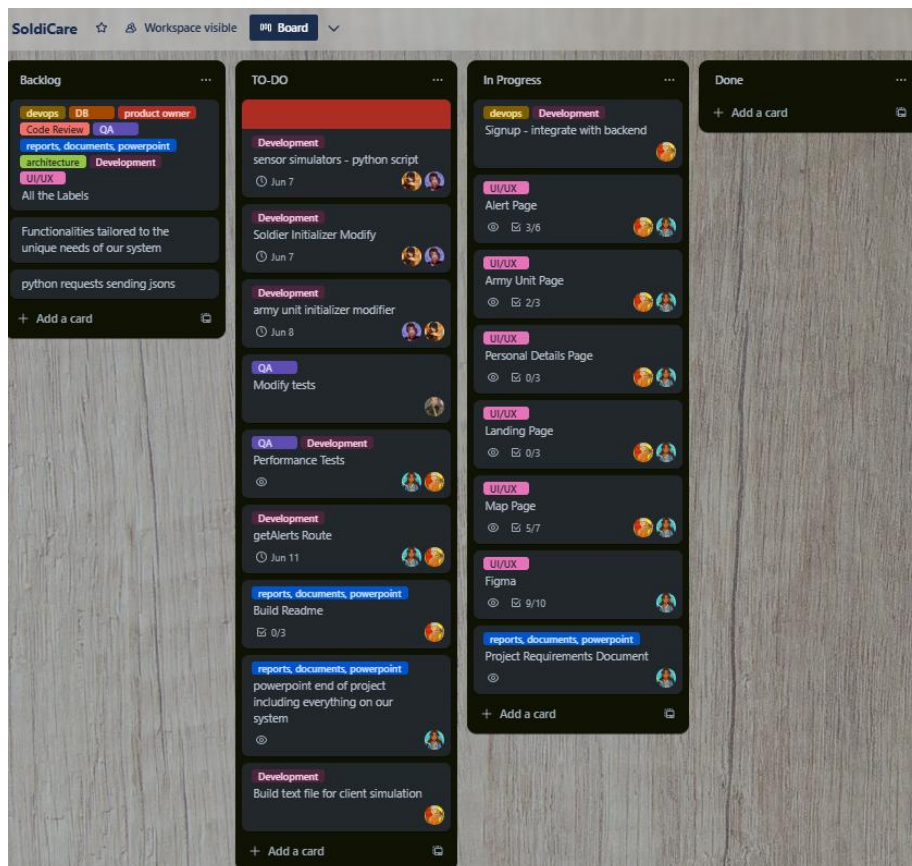
Date: 24/05/2025



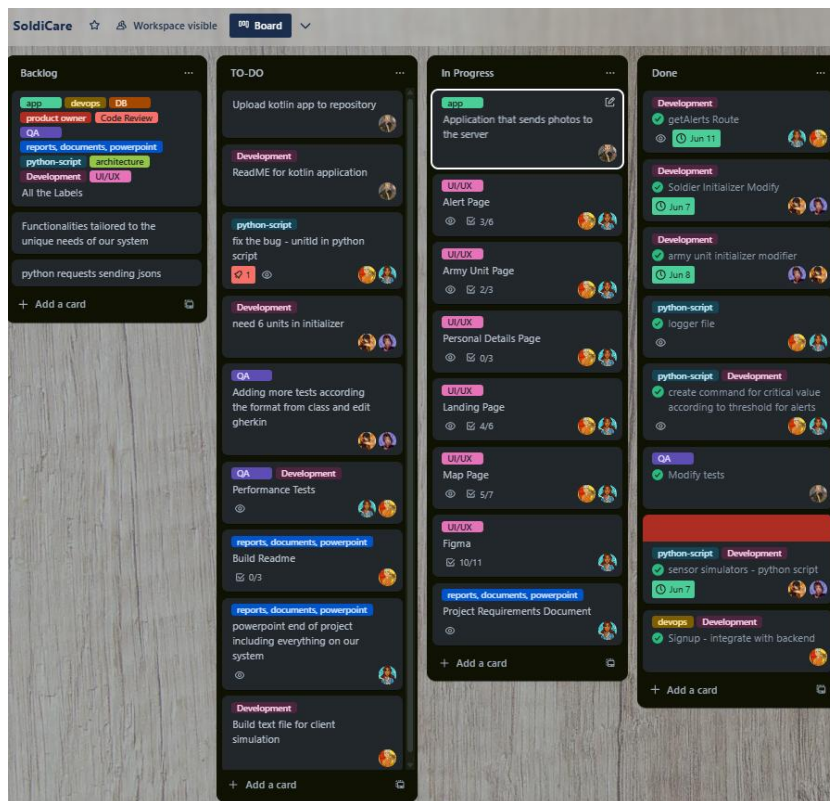


### 6.1.5 Sprint 5

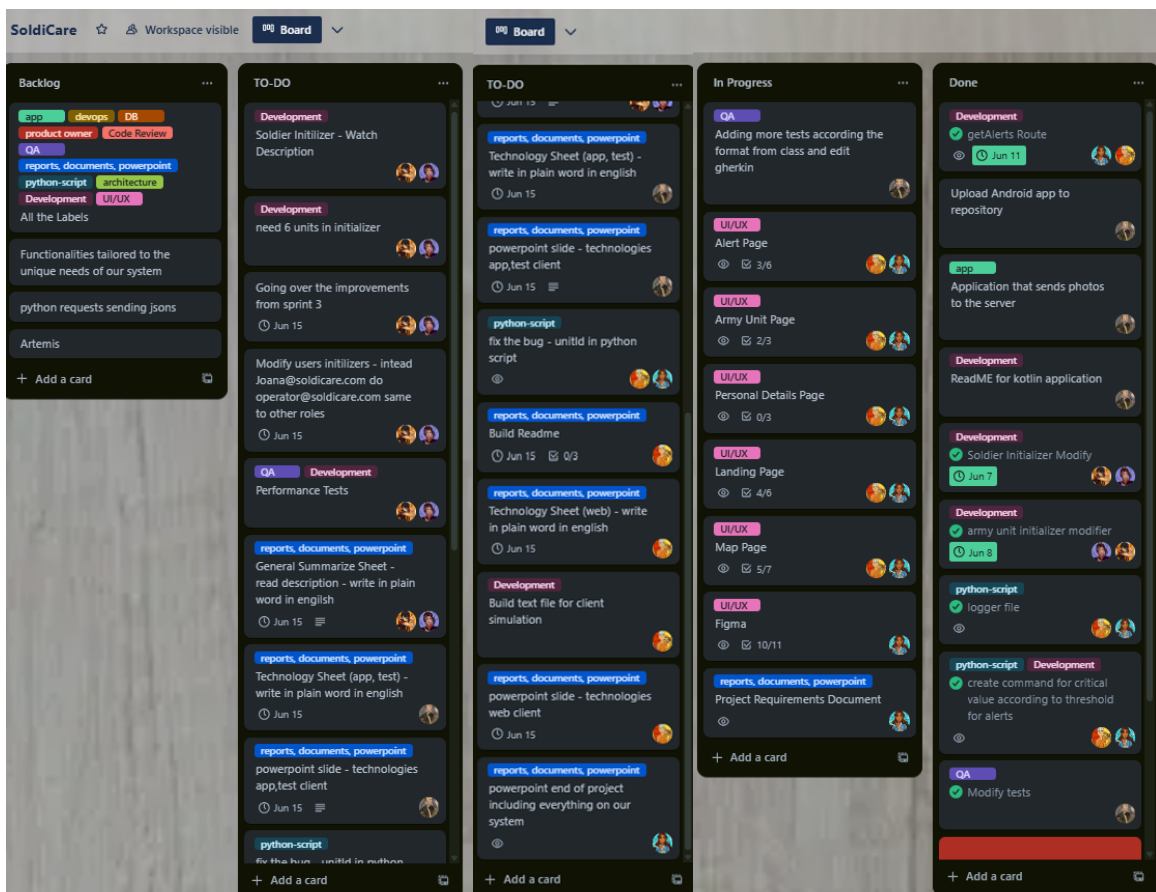
Date: 03/06/2025

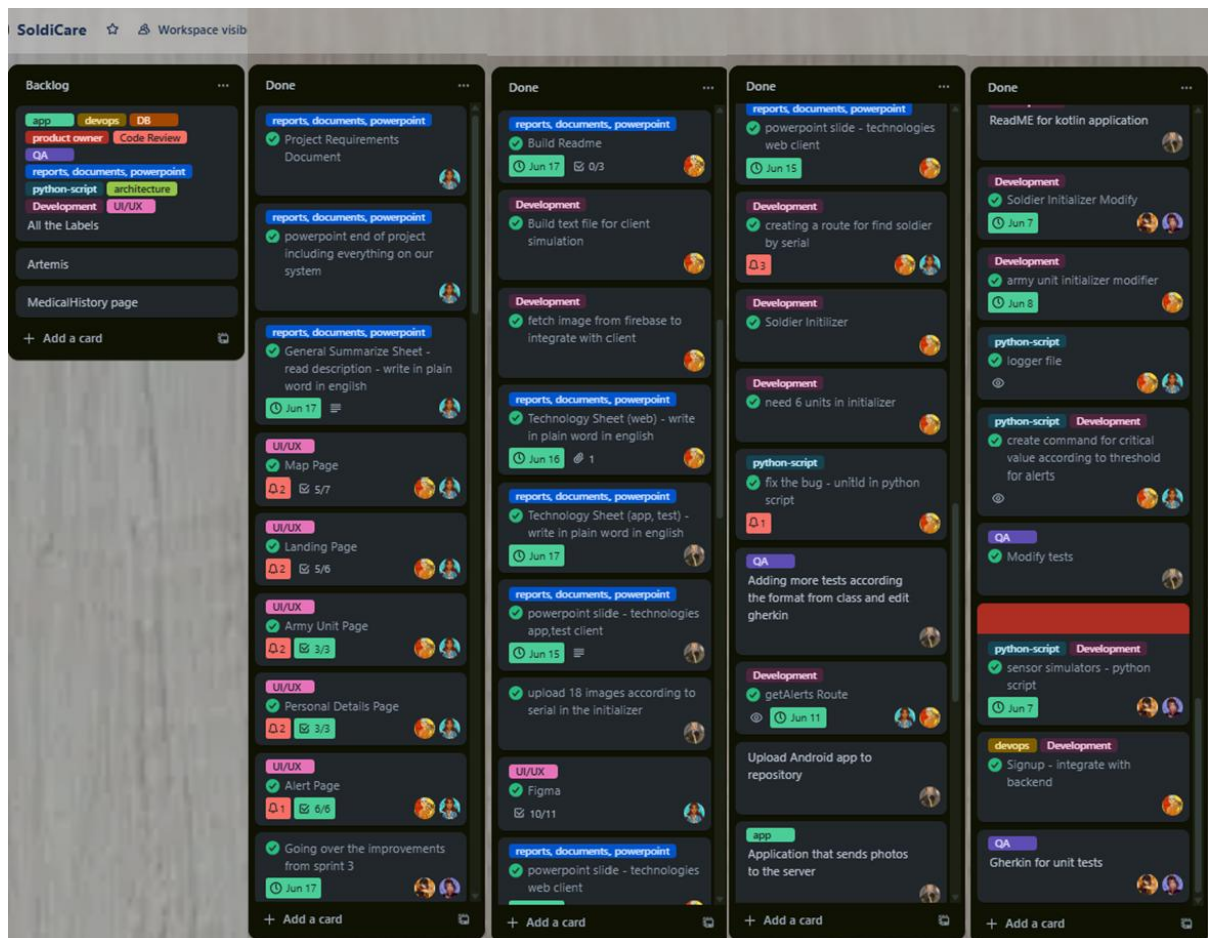






Date: 14/06/2025





## 6.2 Project Retrospective

### 6.2.1 Team Strengths to Maintain

- Effective Kickoff Meetings – Held initial meetings to clarify objectives and distribute tasks efficiently.
- Structured Git workflow – Maintained organized branching and thorough PR reviews for code quality.
- Dividing tasks into sub-teams – Assigned focused tasks to small groups for better efficiency.
- Commitment to deadlines and quality – Met sprint goals while ensuring high development standards.
- Thorough code review processes – Conducted mandatory reviews before merging PRs to prevent issues.



### *6.2.2 Areas for Improvement*

- Strengthening team communication and synchronization – Holding regular meetings to improve coordination and workflow.
- Encouraging proactive engagement and responsibility – Promoting initiative and ownership in task execution.
- Enhancing feedback culture – Creating an open space for constructive discussions and shared insights.
- Improving task execution within timelines – Maintaining steady progress on individual and sub-team tasks, ensuring that all assigned sprint activities are completed on time.
- Fostering innovation and new initiatives – Supporting creative thinking and exploring new approaches for efficiency.

### *6.2.3 Most Enjoyable Aspects*

- Hands-On Learning Experience – Gaining practical experience through a real project, applying theoretical knowledge, and developing new skills.
- Creative Problem-Solving – Tackling technical challenges and finding innovative solutions that made the work engaging and rewarding.
- Building a Functional Product – Seeing a fully operational system designed, developed, and successfully tested by the team.

### *6.2.4 What We Would Do Differently*

- Assigning roles based on prior knowledge and individual expertise.
- Earlier planning of all system components to be implemented – Establishing a comprehensive development roadmap at the project's outset to ensure smoother execution.

### *6.2.5 Remote Collaboration*

- Tools Used
  - Microsoft Teams – Used for team meetings and remote discussions.
  - Trello – Managed task distribution among team members efficiently.
  - Bitbucket & Git – Maintained structured workflow with Pull Requests, branches, and code reviews for each task, alongside separate repositories for different system components.

- Slack – Shared key points from lessons, explanation documents, and technical guides (Git commands, Docker integration, etc.), while receiving notifications on PRs from Git.
  - Google Docs – Facilitated collaborative document sharing and editing.
- Impact of Remote Work – Remote collaboration allowed for flexible communication and task management, ensuring smooth progress despite physical distance. While digital tools streamlined workflow, real-time coordination was sometimes challenging, emphasizing the importance of maintaining structured sync meetings and clear documentation.

## 7. Appendix – External Links

### 7.1 Figma

[Figma Link](#)

### 7.2 System Video

[System Video Link](#)

### 7.3 Bitbucket

[Bitbucket Link](#)